



Sistema Gerenciador Esportivo

Documentação Técnica Completa — TCC ETEC 02/2026

Grupo	Iago Silva · Victor Hugo · Yago Ribeiro · Leonardo Assis · Arthur
Instituição	ETEC — Escola Técnica Estadual
Turma / Ano	02 / 2026
Versão	2.0.0 — Edição Completa com Telas e Banca
Data	02/06/2026
Stack	Java 21 · Spring Boot 3.2.5 · MySQL 8 · HTML/CSS/JS · n8n · Qdrant · OpenAI

Este documento cobre arquitetura, entidades, endpoints, código-fonte, descrição detalhada de cada tela e perguntas para a banca avaliadora.

Sumário

- 1 Visão Geral do Projeto
- 2 Objetivo e Problema Resolvido
- 3 Tecnologias Utilizadas
- 4 Arquitetura do Sistema
- 5 Banco de Dados — Tabelas e Relacionamentos
- 6 Models — Entidades JPA (código completo)
- 7 DAOs — Queries JPQL
- 8 Controllers — Todos os Endpoints REST
- 9 Frontend — Descrição Detalhada de Cada Tela
 - 9.1 login.html
 - 9.2 selecionar-time.html
 - 9.3 dashboard.html
 - 9.4 usuarios.html
 - 9.5 cadastrar-treinador.html
 - 9.6 times.html
 - 9.7 novo-time.html / editar-time.html
 - 9.8 atividades.html
 - 9.9 escalacao.html
 - 9.10 calendario.html
 - 9.11 desempenho.html
 - 9.12 exames.html
 - 9.13 visitantes.html
 - 9.14 assistente.html (novo)
- 10 Regras de Negócio
- 11 Autenticação e Controle de Acesso
- 12 Chat Inteligente — Planejamento Completo
- 13 AssistenteController — Código Completo
- 14 n8n — Fluxos de Automação
- 15 Qdrant — Base Vetorial e RAG
- 16 Docker e Ambiente Local
- 17 Segurança e Melhorias Planejadas
- 18 Como Executar o Projeto
- 19 Perguntas da Banca — Gabarito + Espaço para Respostas

1. Visão Geral do Projeto

O **LAIVY** é um sistema web de gerenciamento esportivo desenvolvido como TCC pela turma 02/2026 da ETEC. Centraliza times, atletas, atividades, desempenho físico, exames médicos, escalações e calendário em uma única plataforma com controle de acesso por perfil.

O sistema é composto por uma API REST em Spring Boot, banco de dados MySQL e um frontend em HTML puro servido pelo próprio servidor. Um chat inteligente com IA está sendo integrado como diferencial do projeto, usando RAG com Qdrant, orquestração com n8n e geração de linguagem com OpenAI.

Módulo	Descrição
Gestão de Usuários	Cadastro completo de atletas, treinadores e admins com foto, dados pessoais, responsáveis e vínculo com time.
Gestão de Times	Criação de times com cor, logo em Base64, modalidade, categoria e treinador responsável.
Atividades	Registro de treinos, jogos, eventos e competições com status, placar e adversário.
Calendário	Visualização em calendário FullCalendar com filtros por tipo de atividade.
Desempenho	Métricas físicas: salto horizontal/vertical, pontuação e histórico por atleta.
Exames	Controle médico: altura, peso, gordura, IMC, validade e status apto/inapto.
Escalação	Montagem de escalação por jogo com quadra SVG, titulares, reservas e pontos marcados.
Visitantes	Registro de visitantes por evento de calendário.
Chat Inteligente	Assistente de IA com RAG (PDF + dados reais) via n8n, Qdrant e OpenAI.

2. Objetivo e Problema Resolvido

Treinadores amadores e escolares gerenciam dados em planilhas desconectadas, sem histórico de desempenho, sem controle médico integrado e sem agenda esportiva centralizada. O LAIVY resolve isso.

Objetivos Específicos

- Centralizar o cadastro de atletas com dados pessoais, foto e contato dos responsáveis.
- Gerenciar múltiplos times com identidade visual (cor, logo, categoria).
- Acompanhar o histórico de desempenho físico de cada atleta ao longo do tempo.
- Controlar o status de aptidão médica via exames com prazo de validade.
- Visualizar o calendário de atividades de forma interativa com filtros por tipo.
- Montar escalações por jogo e registrar pontuação individual de cada jogador.
- Fornecer um assistente de IA que responde sobre o sistema e sobre dados reais.
- Demonstrar domínio técnico avançado para a banca avaliadora do TCC.

3. Tecnologias Utilizadas

Backend

Tecnologia	Uso no Projeto
Java 21	Linguagem principal. Usa records, sealed classes e novos recursos de concorrência.
Spring Boot 3.2.5	Framework para API REST. Auto-configuração, embedded Tomcat.
Spring Data JPA	Abstração de banco com Hibernate. Repositórios com @Query JPQL.
MySQL 8	Banco relacional. ddl-auto=update cria/atualiza tabelas automaticamente.
Maven	Build e gerenciamento de dependências via pom.xml.

Frontend

Tecnologia	Uso no Projeto
HTML5 / CSS3 / JS	Telas estáticas em /static/, sem framework JS.
Bootstrap 5.3.2	Grid, componentes e utilitários CSS.
FullCalendar 6.1.8	Calendário interativo com eventos, filtros e drag-and-drop.
Flatpickr	Date picker nos formulários de atividades e exames.
Google Fonts	Bebas Neue (títulos) + Barlow (corpo).

Chat Inteligente

Tecnologia	Uso no Projeto
n8n	Orquestrador low-code dos fluxos de IA. Docker porta 5678.
OpenAI API	Embeddings (text-embedding-3-small) e respostas (gpt-4o-mini).
Qdrant	Banco vetorial para busca semântica (RAG). Docker porta 6333.
Docker	Containerização de n8n e Qdrant com docker-compose.

4. Arquitetura do Sistema

O LAIVY usa arquitetura em **3 camadas**: Model → DAO → Controller, sem Service layer. O frontend se comunica com os controllers via **fetch()** + **JSON**. O Spring Boot serve tanto a API quanto os arquivos estáticos.

Fluxo completo de requisição

```
Browser (HTML/JS)
■■■> fetch('http://localhost:8080/usuarios', { method: 'GET' })
■■■> UsuarioController (@RestController @GetMapping)
■■■> IUsuario.findAll() (JpaRepository)
■■■> Hibernate gera SQL → MySQL executa
■■■> List<Usuario> serializado para JSON
■■■> ResponseEntity.ok(lista) → 200 OK
■■■> JavaScript recebe array JSON → renderiza cards/tabela
```

Estrutura de Pacotes

```
src/main/
■■■ iava/br/com/gerencador/projeto/
■ ■■■ ProjetoApplication.java ← @SpringBootApplication. main()
■ ■■■ model/ ← Entidades JPA (@Entity)
■ ■ ■■■ Usuario.java
■ ■ ■■■ Time.java
■ ■ ■■■ Atividade.java
■ ■ ■■■ Calendario.java
■ ■ ■■■ Desempenho.java
■ ■ ■■■ Exame.java
■ ■ ■■■ Escalacao.java
■ ■ ■■■ Visitante.java
■ ■■■ DAO/ ← Repositórios (JpaRepository + @Query)
■ ■ ■■■ IUsuario.java
■ ■ ■■■ TimeDAO.java
■ ■ ■■■ AtividadeDAO.java
■ ■ ■■■ CalendarioDAO.java
■ ■ ■■■ DesempenhoDAO.java
■ ■ ■■■ ExameDAO.java
■ ■ ■■■ EscalacaoDAO.java
■ ■ ■■■ VisitanteDAO.java
■ ■■■ controller/ ← REST Controllers
■ ■■■ UsuarioController.java
■ ■■■ TimeController.java
■ ■■■ AtividadeController.java
■ ■■■ CalendarioController.java
■ ■■■ DesempenhoController.java
■ ■■■ ExameController.java
■ ■■■ EscalacaoController.java
■ ■■■ VisitanteController.java
■ ■■■ AssistenteController.java ← NOVO (chat IA)
■■■ resources/
■■■ application.properties
■■■ static/ ← HTML, CSS, JS servidos em /
■■■ login.html
■■■ dashboard.html
■■■ ... (14 telas)
```

5. Banco de Dados — Tabelas e Relacionamentos

Schema: **sistema_esportivo**. As tabelas são gerenciadas pelo Hibernate com **ddl-auto=update** — criadas/atualizadas automaticamente na inicialização do sistema.

Tabela	Registra	PKs e FKs principais
tbl_usuario	Todos os usuários do sistema	id_usuario (PK), id_time (FK→tbl_time)
tbl_time	Times esportivos	id_time (PK), id_treinador (FK→tbl_usuario)
tbl_atividade	Treinos, jogos, eventos	id_atividade (PK), id_usuario (FK), id_time (FK)
tbl_calendario	Resultados de atividades	id_calendario (PK), id_usuario (FK), id_atividade (FK)
tbl_desempenho	Métricas físicas por atleta	id_desempenho (PK), id_usuario (FK)
tbl_exame	Exames médicos dos atletas	id_exame (PK), id_usuario (FK)
tbl_escalacao	Escalação por jogo	id_escalacao (PK), id_atividade (FK), id_usuario (FK)
tbl_visitante	Visitantes por evento	id_visitante (PK), id_calendario (FK)

Relacionamentos entre Entidades

De	Para	Tipo	Descrição
Usuario	Time	ManyToOne	Atleta pertence a um time
Time	Usuario (treinador)	ManyToOne	Time tem um treinador responsável
Atividade	Usuario	ManyToOne	Atividade criada por um usuário
Atividade	Time	ManyToOne	Atividade pertence a um time
Atividade	Calendario	OneToMany	Uma atividade tem N calendários
Calendario	Visitante	OneToMany	Um evento tem N visitantes
Escalacao	Atividade	ManyToOne	Escalação é de um jogo específico
Escalacao	Usuario	ManyToOne	Escalação de um jogador
Exame	Usuario	ManyToOne	Exame pertence a um atleta
Desempenho	Usuario	ManyToOne	Desempenho de um atleta

6. Models — Entidades JPA

Todas as entidades: `@Entity @Table`, PK com `@Id @GeneratedValue(IDENTITY)`, relacionamentos com `@ManyToOne(fetch=EAGER)` e `@JsonIgnoreProperties` para evitar loop de serialização JSON.

6.1 Usuario.java

Campo	Tipo Java	Coluna MySQL	Observação
<code>id_usuario</code>	Integer	PK AUTO_INCREMENT	Chave primária
<code>nome_usuario</code>	String	VARCHAR(100) NOT NULL	Nome completo
<code>email_usuario</code>	String	VARCHAR(100) UNIQUE NOT NULL	Email único
<code>login_usuario</code>	String	VARCHAR(45) UNIQUE NOT NULL	Login de acesso
<code>senha_usuario</code>	String	VARCHAR(100) NOT NULL	Texto plano — BCrypt planejado
<code>primeiro_login</code>	Integer	INT	1=trocar senha, 0=já redefiniu
<code>tipo_usuario</code>	Integer	INT NOT NULL	1=Admin, 2=Treinador, 3=Atleta
<code>data_nasc_usuario</code>	LocalDate	DATE	@JsonFormat(pattern='yyyy-MM-dd')
<code>foto_usuario</code>	String	MEDIUMTEXT	Base64 da foto
<code>endereco/bairro/cep/cidade/estado</code>	String	VARCHAR	Endereço completo
<code>cel_usuario / cpf_usuario</code>	String	VARCHAR	Contato e documento
<code>rg/cpf/cel_responsavel_1/2</code>	String	VARCHAR	Dados dos responsáveis
<code>time</code>	Time	FK id_time	@ManyToOne @JsonIgnoreProperties

6.2 Time.java

Campo	Tipo	Observação
<code>id_time</code>	Integer	PK
<code>nome_time</code>	String(100)	NOT NULL
<code>modalidade_time</code>	String(60)	Ex: Basquete, Futebol
<code>categoria_time</code>	String(60)	Sub-13, Sub-15, Sub-17, Adulto
<code>cor_time</code>	String(20)	Hex: #00C97A
<code>descricao_time</code>	TEXT	Texto livre
<code>ativo_time</code>	Integer	1=ativo, 0=inativo
<code>logo_time</code>	MEDIUMTEXT	Base64 da logo

Campo	Tipo	Observação
treinador	Usuario	@ManyToOne — treinador responsável
membros	List	@OneToMany @JsonIgnore

6.3 Atividade.java

Campo	Tipo	Observação
id_atividade	Integer	PK
nome_atividade	String(100)	NOT NULL
lugar_atividade	String(60)	Local
data_atividade	LocalDateTime	Data e hora
status_atividade	Integer	0=Cancelado, 1=Ativo, 2=Finalizado
tipo_atividade	Integer	1=Treino, 2=Jogo, 3=Evento, 4=Competição
time_oponente	String(100)	Adversário (jogos/competições)
placar_pro / placar_contra	Integer	Resultado do jogo
usuario	Usuario	@ManyToOne criador
time	Time	@ManyToOne time
calendarios	List	@OneToMany cascade=ALL

6.4 Escalacao.java

Campo	Tipo	Observação
id_escalacao	Integer	PK
numero_camisa	Integer	Número da camisa
posicao	String(50)	Armador, Ala, Pivô, etc.
titular	Integer	1=Titular, 0=Reserva
pontos_jogo	Integer	Pontos marcados neste jogo
atividade	Atividade	@ManyToOne
usuario	Usuario	@ManyToOne

7. DAOs — Queries JPQL

Todos estendem **JpaRepository<Entity, Integer>**. Queries customizadas usam **@Query** com JPQL (não SQL nativo), operando sobre classes Java, não tabelas.

IUsuario.java

```
Optional<Usuario> autenticar(@Param("login") String login,
    @Param("senha") String senha,
    @Param("tipo") Integer tipo):
// @Querv: WHERE login usuario=:login AND senha usuario=:senha AND tipo usuario=:tipo
List<Usuario> buscarPorTime(@Param("idTime") Integer idTime):
// @Querv: WHERE u.time.id time = :idTime
List<Usuario> buscarPorTimeTipo(@Param("idTime") Integer idTime,
    @Param("tipo") Integer tipo):
// @Query: WHERE u.time.id_time = :idTime AND u.tipo_usuario = :tipo
```

AtividadeDAO.java

```
List<Atividade> buscarPorUsuario(@Param("id") Integer id):
// ORDER BY data atividade DESC
List<Atividade> buscarPorTipo(@Param("tipo") Integer tipo):
List<Atividade> buscarPorTime(@Param("idTime") Integer idTime):
// ORDER BY data_atividade DESC
```

DesempenhoDAO.java

```
List<Desempenho> buscarPorUsuario(@Param("id") Integer id):
// ORDER BY data desempenho DESC
List<Desempenho> buscarPorTime(@Param("idTime") Integer idTime):
// WHERE d.usuario.time.id_time = :idTime ORDER BY data_desempenho DESC
```

TimeDAO.java

```
List<Time> findBvAtivo time(@Param("ativo") Integer ativo):
// @Querv explícita – Spring Data não parseia "ativo time" por convenção de nome
List<Time> findBvTreinador(@Param("idTreinador") Integer idTreinador):
List<Time> buscarPorNome(@Param("nome") String nome):
// LIKE LOWER(CONCAT('%', :nome, '%')) – busca case-insensitive
```

EscalacaoDAO.java

```
List<Escalacao> buscarPorAtividade(@Param("id") Integer id):
// WHERE e.atividade.id_atividade = :id
```

8. Controllers — Todos os Endpoints REST

Todos: **@RestController @CrossOrigin("*") @RequestMapping**. Respostas em **ResponseEntity** com HTTP status semântico. Atualizações parciais: só aplica campos não-null do body recebido.

USUÁRIOS

Método	Endpoint	Descrição
GET	/usuarios	Lista todos. Query param opcional: ?idTime=N
GET	/usuarios/{id}	Busca por ID. 404 se não encontrado
GET	/usuarios/{id}/exames	Lista exames do usuário
GET	/usuarios/treinadores	Lista tipo_usuario <= 2 (admin + treinador)
POST	/usuarios	Cria usuário. Resolve FK time antes de salvar
POST	/usuarios/login	Autentica: login+senha+tipo. 200 ou 401
PUT	/usuarios/{id}	Atualiza parcialmente. Só aplica campos não-null
PUT	/usuarios/{id}/senha	Troca senha + seta primeiro_login=0
DELETE	/usuarios/{id}	Remove. 404 se não existe

TIMES

Método	Endpoint	Descrição
GET	/times	Lista todos os times
GET	/times/ativos	Filtra ativo_time=1
GET	/times/{id}	Busca por ID
GET	/times/{id}/membros	Lista usuarios com id_time = id
POST	/times	Cria. ativo_time=1 por padrão se null
PUT	/times/{id}	Atualiza parcialmente
PUT	/times/{id}/membros/{idUsr}	Vincula usuário ao time
DELETE	/times/{id}/membros/{idUsr}	Seta id_time=null no usuário
DELETE	/times/{id}	Remove time

ATIVIDADES

Método	Endpoint	Descrição
GET	/atividades	Lista. ?idTime= filtra por time
GET	/atividades/{id}	Busca por ID
GET	/atividades/usuario/{id}	Por criador, ordenado por data DESC
GET	/atividades/tipo/{tipo}	1=Treino 2=Jogo 3=Evento 4=Competição

Método	Endpoint	Descrição
GET	/atividades/time/{idTime}	Por time, ordenado por data DESC
POST	/atividades	Cria. Resolve FKs usuario e time
PUT	/atividades/{id}	Atualiza parcialmente
DELETE	/atividades/{id}	Remove

CALENDÁRIO / DESEMPENHO / EXAMES / ESCALAÇÃO / VISITANTES

Método	Endpoint	Descrição
GET	/calendarios	Lista todos os calendários
GET	/calendarios/usuario/{id}	Calendários de um usuário
GET	/calendarios/atividade/{id}	Calendários de uma atividade
POST	/calendarios	Cria. Resolve FKs usuario e atividade
PUT	/calendarios/{id}	Atualiza
DELETE	/calendarios/{id}	Remove
GET	/desempenhos	Lista todos
GET	/desempenhos/usuario/{id}	Por atleta, ordenado por data DESC
POST	/desempenhos	Cria. Resolve FK usuario
PUT	/desempenhos/{id}	Atualiza
DELETE	/desempenhos/{id}	Remove
GET	/exames	Lista todos
GET	/exames/usuario/{id}	Por atleta
POST	/exames	Cria
PUT	/exames/{id}	Atualiza
DELETE	/exames/{id}	Remove
GET	/escalacoes/atividade/{id}	Escalação de um jogo
POST	/escalacoes	Adiciona jogador à escalação
PUT	/escalacoes/{id}	Atualiza pontos/posição/camisa/titular
DELETE	/escalacoes/{id}	Remove jogador da escalação
GET	/visitantes	Lista todos
GET	/visitantes/calendario/{id}	Por evento
POST	/visitantes	Cria
PUT	/visitantes/{id}	Atualiza
DELETE	/visitantes/{id}	Remove

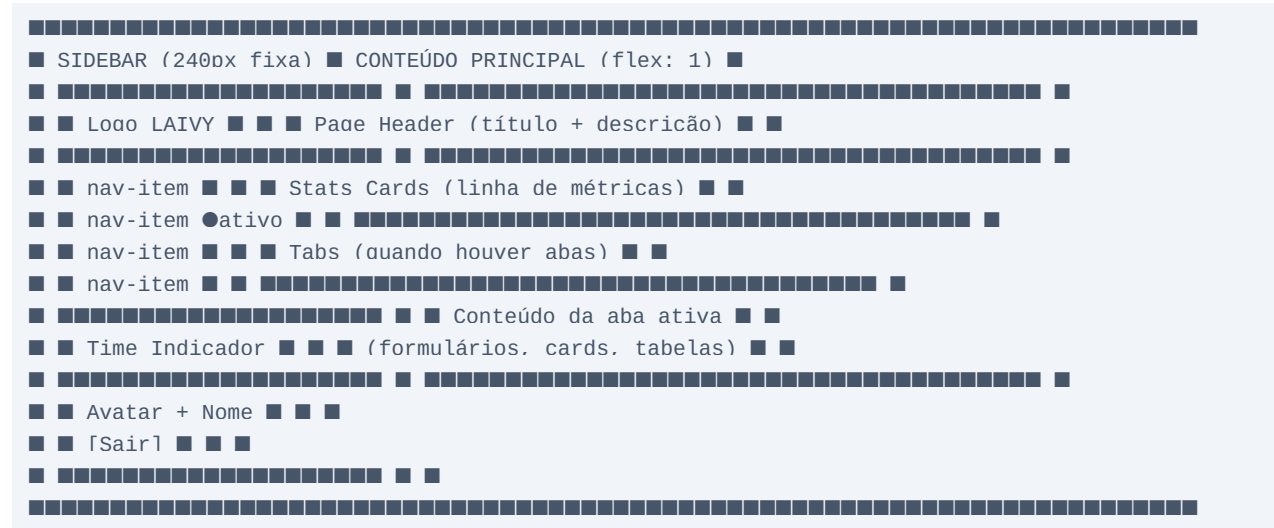
ASSISTENTE IA

Método	Endpoint	Descrição
POST	/assistente/chat	Envia pergunta ao n8n. Retorna resposta da IA
GET	/assistente/resumo-geral	Totais: usuários, atletas, times, atividades
GET	/assistente/resumo-time	Resumo do time: membros, modalidade, treinador
GET	/assistente/ranking-desempenho	Ranking por média de pontuação
GET	/assistente/ranking-pontos	Ranking de pontos por jogo
GET	/assistente/proximas-atividades	Próximas 5 atividades ativas

9. Frontend — Descrição Detalhada de Cada Tela

O frontend é HTML puro servido pelo Spring Boot. Toda a lógica de sessão usa **localStorage**. O design system é consistente em todas as telas: sidebar fixa, fundo #0A1628, verde #00C97A, fontes Bebas Neue + Barlow.

Padrão de Layout (todas as telas logadas)



9.1 login.html



Campos:

- **Login** — input text, placeholder 'seu login', required.
- **Senha** — input password, required.
- **Tipo de usuário** — select com opções: Administrador (1), Treinador (2), Atleta (3).
- **Botão ENTRAR** — chama POST /usuarios/login. Em caso de erro, exibe mensagem vermelha inline.

Comportamento após login:

- Salva o objeto do usuário em **localStorage.setItem('usuarioLogado', JSON.stringify(usuario))**.
- Se tipo=3 (atleta): redireciona direto para **dashboard.html** (atleta já tem time vinculado).
- Se tipo=1 ou 2: redireciona para **selecionar-time.html**.
- Se primeiro_login=1: redireciona para tela de troca de senha antes do dashboard.
- Animação de fundo: dois círculos com radial-gradient pulsando via @keyframes pulse.

- Em caso de falha (401 ou rede): alerta amigável, não apaga os campos preenchidos.

9.2 seleccionar-time.html

[illegible]

- Exibida apenas para **Admin (tipo=1)** e **Treinador (tipo=2)**. Atleta vai direto ao dashboard.
- Carrega todos os times ativos via **GET /times/ativos**.
- Cada card exibe: banner colorido (cor do time), logo, nome, modalidade/categoria, 3 stats (membros, atividades, exames), nome do treinador.
- Ao clicar no card: salva em **localStorage.setItem('timeSelecionado', JSON.stringify(time))** e redireciona para **dashboard.html**.
- Seta animada (→) aparece no hover do card.
- Estado de loading com spinner enquanto carrega os times.
- Se nenhum time ativo existir: exibe mensagem orientando o admin a criar um time.

9.3 dashboard.html

```
■ ■ SIDEBAR ■ TOPBAR: título + relógio em tempo real ■  
■ ■ Card boas-vindas (nome + time + data grande) ■  
■ ■ [Atletas] [Times] [Atividades] [Exames] ■  
■ ■ stat card stat card stat card stat card ■  
■ ■ Linha 2 de cards (exames vencidos, próximos) ■  
■ ■ Atividades recentes (lista) ■
```

- **Relógio em tempo real:** atualizado a cada segundo via setInterval.
- **Card de boas-vindas:** exibe nome do usuário logado, data/mês e time selecionado.
- **Stats cards:** 4 cards com totais — atletas, times, atividades, exames. Cada um leva à tela correspondente ao clicar.
- Carrega dados via chamadas paralelas às APIs: GET /usuarios, /times, /atividades, /exames.
- Comportamento por perfil: atleta vê só dados do próprio time; admin vê dados globais.

9.4 usuarios.html

- Cards de times com banner na cor do time, logo, nome, modalidade, categoria.
- Botão **Editar**: leva para `editar-time.html` com id do time na URL (`?id=N`).
- Botão **Excluir**: confirmação antes de `DELETE /times/{id}`.
- **Aba Cadastrar**: abre formulário inline (mesmo layout de `novo-time.html` mas embutido).
- **Color picker**: `input type=color` + swatch visual. Cor usada no banner e no badge.
- **Upload de logo**: `input type=file`, preview circular, converte para Base64.
- **Seleção de treinador**: grade de cards com avatar, nome e tipo. Marca com ✓ ao selecionar.

- **Preview ao vivo:** à medida que o usuário digita nome, seleciona cor ou faz upload de logo, o card de preview à direita atualiza em tempo real via eventos `input/change`.
- **editar-time.html:** carrega dados do time via `GET /times/{id}` (id vem da URL `?id=N`) e preenche o formulário. Ao salvar: `PUT /times/{id}`.
- **novo-time.html:** formulário em branco. Ao salvar: `POST /times`.
- **Treinador sticky:** preview fica posicionado com `position:sticky; top:24px`.
- Validação de campo obrigatório: nome do time. Botão Salvar desabilitado até preenchimento.

- Cards com border-left colorida por tipo: verde=treino, amarelo=jogo, roxo=evento, azul=competição.
- Badge de tipo e badge de status em cada card.
- Botão **Escalação** em cards de jogo: leva para `escalacao.html?id={id}`.
- Busca filtra por nome da atividade (client-side).

Aba Calendário:

- FullCalendar 6.1 em modo mensal. Eventos carregados de `GET /atividades?idTime=N`.
- Filtros por tipo: botões toggle que mostram/ocultam categorias de eventos.
- Ao clicar em um evento: exibe modal com detalhes da atividade.

Formulário de Cadastro:

- **Nome**, **local**, **data/hora**, **tipo** (select), **status** (select).
- Se tipo = Jogo ou Competição: aparece campo `time oponente`, `placar pro` e `placar contra`.
- Ao salvar: `POST /atividades` com `{ nome, lugar, data, tipo, status, time: {id_time} }`.

9.9 escalacao.html



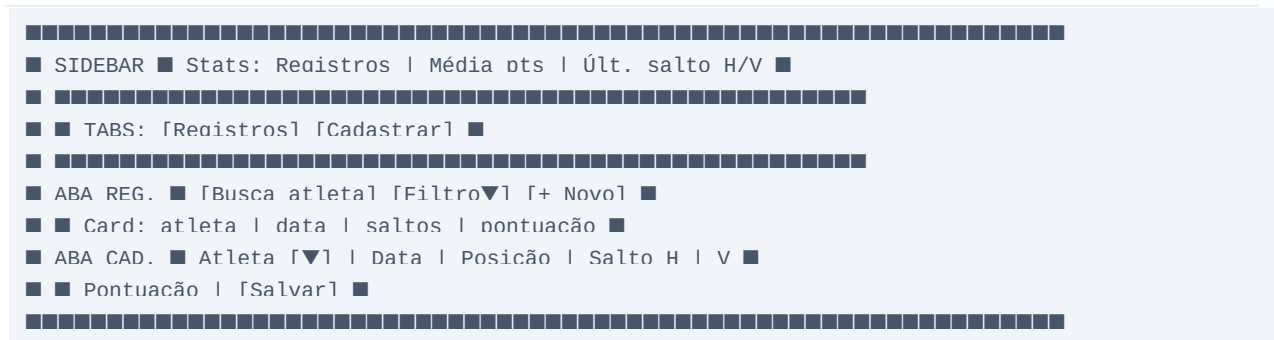
- **Seletor de jogo**: dropdown que carrega atividades do `tipo=2` (jogo) do time selecionado.
- **Quadra SVG**: desenho de quadra de basquete em SVG com posições marcadas.
- Jogadores titulares aparecem posicionados na quadra de acordo com a posição cadastrada.
- **Painel direito**: lista de titulares e reservas com avatar, nome e posição.
- Botão **+ Adicionar Titular / + Adicionar Reserva**: abre modal de seleção de atletas do time.
- Modal de seleção: lista atletas do time não escalados. Ao escolher: seleciona posição e número de camisa.
- Botão **Remove**: `DELETE /escalacoes/{id}`.
- Campo **Pontos no jogo**: editável inline via `PUT /escalacoes/{id}` com `{ pontos_jogo: N }`.
- Stats no topo: contadores de titulares, reservas e total de pontos marcados.

9.10 calendario.html

- Calendário FullCalendar em tela cheia com filtros por tipo de atividade.
- Botões toggle: [Todos] [Treino] [Jogo] [Evento] [Competição]. Cada um com cor correspondente.
- Stats acima do calendário: total de eventos no mês, por tipo.
- Ao clicar num evento: abre modal com detalhes — nome, tipo, status, local, data, placar (se jogo).
- Carrega atividades via `GET /atividades?idTime=N` e monta eventos no formato FullCalendar.

- Cores dos eventos: verde=treino, amarelo=jogo, roxo=evento, azul=competição.

9.11 desempenho.html



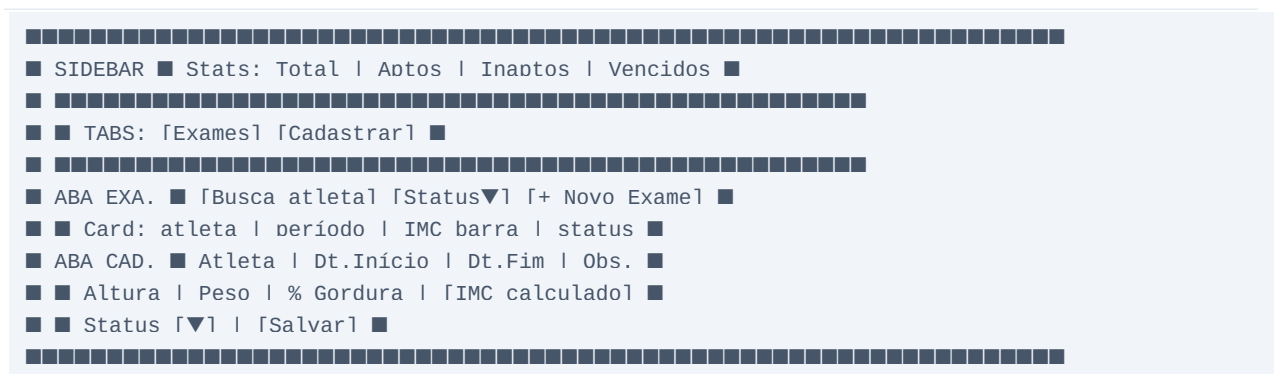
Aba Registros:

- Cards de desempenho com: nome do atleta, data, salto horizontal (m), salto vertical (m), pontuação.
- Botões editar e excluir em cada card.
- Busca por nome de atleta (client-side).

Aba Cadastrar:

- **Atleta:** select populado com GET /usuarios?idTime=N, filtrado por tipo=3.
- **Data:** date picker.
- **Posição/número:** campo numérico.
- **Salto horizontal e vertical:** campos numéricos em metros.
- **Pontuação:** número inteiro.
- Ao salvar: POST /desempenhos com { usuario: {id}, data, salto_horizontal, salto_vertical, pontuacao }.

9.12 exams.html



Aba Exams:

- Cards com: nome do atleta, período (data início → fim), badge de status (verde=Apto, vermelho=Inapto).
- **Barra de IMC:** calculada a partir de peso/altura, com cor indicativa (verde/amarelo/vermelho).
- Filtro por status: Todos / Aptos / Inaptos.

Aba Cadastrar:

- **Atleta:** select com atletas do time.

- **Data início e fim:** date pickers para validade do exame.
- **Altura (m), Peso (kg), Taxa de gordura (%):** campos numéricos.
- **IMC calculado:** aparece automaticamente ao preencher altura e peso ($\text{peso}/\text{altura}^2$).
- **Status:** select 0=Inapto / 1=Apto.
- **Observações:** textarea para notas médicas.
- Dados médicos individuais **não são expostos pelo chat** por privacidade.

9.13 visitantes.html

- Registro de visitantes presentes em eventos do calendário.
- Stats: total de visitantes, municípios distintos.
- **Aba Lista**: busca por nome, filtro por evento. Cards com nome e município.
- **Aba Cadastrar**: campos nome, município e seleção do evento (calendário) ao qual pertence.
- Select de eventos carregado via GET /calendarios.
- Ao salvar: POST /visitantes com { nome, municipio, calendario: {id_calendario} }.

9.14 assistente.html (nova tela)



- **Perguntas prontas:** chips clicáveis que preenchem e enviam a pergunta automaticamente.
- **Histórico:** mensagens do usuário alinhadas à direita (verde), assistente à esquerda (azul).
- **Input:** textarea com envio por Enter ou botão. Desabilitado enquanto aguarda resposta.
- **Loading state:** indicador de digitação (...) enquanto a IA processa.
- **Mini-chat flutuante:** botão  fixo em todas as telas logadas (position:fixed, bottom-right). Clique abre um painel flutuante com o chat resumido. Link 'Abrir completo' leva ao assistente.html.
- **Erro de rede:** mensagem amigável se n8n ou OpenAI estiver indisponível.
- Ao enviar: POST /assistente/chat com { pergunta, idUsuario, idTime, tipo_usuario }.

10. Regras de Negócio

Perfis

- tipo_usuario: 1=Admin (acesso total), 2=Treinador (gerencia time), 3=Atleta (visualiza). Controle feito no frontend via localStorage e no backend via lógica de filtro.

Times

- ativo_time=0 oculta o time da seleção. Um atleta pertence a no máximo 1 time por vez. Um time tem exatamente 1 treinador responsável.

Atividades

- tipo: 1=Treino, 2=Jogo, 3=Evento, 4=Competição. status: 0=Cancelado, 1=Ativo, 2=Finalizado. Campos de placar e oponente só fazem sentido em tipo 2 e 4.

Primeiro Login

- Ao criar usuário pelo admin: primeiro_login=1. Sistema redireciona para troca obrigatória de senha. PUT /usuarios/{id}/senha seta primeiro_login=0.

Escalação

- Uma escalação por atividade/jogo. titular=1/0. pontos_jogo atualizável via PUT após o jogo. Quadra exibe posições: PG, SG, SF, PF, C (basquete).

Exames

- status_usuario=0=Inapto deve impedir participação em atividades. Dados médicos individuais (peso, altura, gordura, observações) são confidenciais — o chat não deve expor esses dados nem em resumos.

Chat

- Respeita perfil: admin vê tudo, treinador vê time dele, atleta vê dados próprios. Dados sensíveis nunca retornados: senha, CPF, telefone, endereço, dados médicos individuais.

11. Autenticação e Controle de Acesso

Autenticação via POST /usuarios/login. Sem JWT nem Spring Security. Sessão mantida em localStorage do navegador.

Request — POST /usuarios/login

```
{ "login_usuario": "treinador", "senha_usuario": "1234", "tipo_usuario": 2 }
```

Response 200 OK

```
{ "id_usuario": 2, "nome_usuario": "Carlos", "tipo_usuario": 2, "time": {...} }
```

- **401 Unauthorized:** credenciais inválidas.
- **primeiro_login=1:** sistema redireciona para tela de troca de senha.
- Sessão salva em localStorage como JSON. Lida em toda tela via `JSON.parse(localStorage.getItem('usuarioLogado'))`.

Perfil	Login	Senha	tipo_usuario
Administrador	admin	admin	1
Treinador	treinador	1234	2
Atleta	atleta	1234	3

■ ■ **ATENÇÃO** — Senha em texto plano. Melhoria planejada: `BCryptPasswordEncoder` do Spring Security.

12. Chat Inteligente — Planejamento Completo

Nova integração adicionada ao LAIVY sem substituir nenhuma tela existente. Responde usando PDF (RAG) e dados reais (APIs).

Fluxo completo

```
Usuário digita pergunta
→ POST /assistente/chat (Spring Boot)
→ valida usuário (localStorage) + perfil
→ adiciona header X-Token
→ POST webhook n8n
→ valida X-Token
→ classifica pergunta (OpenAI)
→ se sobre PDF: embed pergunta → busca Odrant → retorna chunks
→ se sobre dados: GET /assistente/resumo-* (LAIVY APIs)
→ monta prompt: contexto + pergunta + perfil + restrições
→ OpenAI gpt-4o-mini gera resposta
→ retorna JSON { resposta, fonte }
→ Spring Boot retorna ao frontend
→ Chat exibe resposta formatada
```

Endpoint	Retorna	Usado pelo chat quando
/assistente/resumo-geral	Totais globais	Admin pergunta sobre o sistema
/assistente/resumo-time	Dados do time	Treinador/atleta pergunta sobre o time
/assistente/ranking-desempenho	Ranking por pontuação	'Quem tem melhor desempenho?'
/assistente/ranking-pontos	Ranking por jogo	'Quem marcou mais pontos no jogo X?'
/assistente/proximas-atividades	Próximas 5 atividades	'Quais são as próximas atividades?'

13. AssistenteController — Código Completo

AssistenteController.java

```
@RestController
@CrossOrigin(origins = "*")
@RequestMapping("/assistente")
public class AssistenteController {
    @Value("${assistente.n8n.webhook-url}")
    private String webhookUrl;
    @Value("${assistente.n8n.token}")
    private String token;
    @Autowired private IUsuario usuarioDAO;
    @Autowired private AtividadeDAO atividadeDAO;
    @Autowired private DesempenhoDAO desempenhoDAO;
    @Autowired private EscalacaoDAO escalacaoDAO;
    @Autowired private TimeDAO timeDAO;
    private final RestTemplate restTemplate = new RestTemplate();
    @PostMapping("/chat")
    public ResponseEntity<Map<String, Object>> chat(
        @RequestBody Map<String, Object> body) {
        try {
            HttpHeaders h = new HttpHeaders();
            h.setContentType(MediaType.APPLICATION_JSON);
            h.set("X-Token", token);
            ResponseEntity<Map> resp = restTemplate.postForEntity(
                webhookUrl, new HttpEntity<>(body, h), Map.class);
            return ResponseEntity.ok(resp.getBody());
        } catch (Exception e) {
            return ResponseEntity.status(503)
                .body(Map.of("erro", "Assistente indisponível no momento."));
        }
    }
    @GetMapping("/resumo-geral")
    public ResponseEntity<Map<String, Object>> resumoGeral() {
        List<Usuario> todos = usuarioDAO.findAll();
        return ResponseEntity.ok(Map.of(
            "total usuarios", todos.size(),
            "total atletas", todos.stream().filter(u->u.getTipo_usuario()==3).count(),
            "total treinadores", todos.stream().filter(u->u.getTipo_usuario()==2).count(),
            "total times", timeDAO.count(),
            "total atividades", atividadeDAO.count()
        ));
    }
    @GetMapping("/resumo-time")
    public ResponseEntity<Map<String, Object>> resumoTime(
        @RequestParam Integer idTime) {
        Map<String, Object> r = new HashMap<>();
        timeDAO.findById(idTime).ifPresent(t -> {
            r.put("time", t.getNome_time());
            r.put("modalidade", t.getModalidade_time());
            r.put("categoria", t.getCategoria_time());
            r.put("membros", t.getMembros().size());
            r.put("treinador", t.getTreinador() != null
                ? t.getTreinador().getNome_usuario() : "N/A");
        });
    }
}
```



```

}):
return ResponseEntity.ok(r):
}

@GetMapping("/ranking-desempenho")
public ResponseEntity<List<Map<String, Object>>> rankingDesempenho(
    @RequestParam Integer idTime) {
    return ResponseEntity.ok(
        desempenhoDAO.buscarPorTime(idTime).stream()
            .collect(java.util.stream.Collectors.groupingBy(
                d -> d.getUsuario().getNome usuario()))
            .entrySet().stream()
            .map(e -> Map.<String, Object>.of(
                "atleta", e.getKey(),
                "media", e.getValue().stream()
                    .mapToInt(d->d.getPontuacao()!=null?d.getPontuacao():0)
                    .average().orElse(0),
                "avaliacoes", e.getValue().size()))
            .sorted((a,b)->Double.compare(
                (Double)b.get("media"), (Double)a.get("media")))
            .collect(java.util.stream.Collectors.toList()));
    }

@GetMapping("/ranking-pontos")
public ResponseEntity<List<Map<String, Object>>> rankingPontos(
    @RequestParam Integer idAtividade) {
    return ResponseEntity.ok(
        escalacaoDAO.buscarPorAtividade(idAtividade).stream()
            .filter(e->e.getPontos !=null)
            .sorted((a,b)->b.getPontos !=null?a.getPontos !=null)
            .map(e->Map.of(
                "atleta", e.getUsuario().getNome usuario(),
                "pontos", e.getPontos !=null,
                "posicao", e.getPosicao()!=null?e.getPosicao():""))
            .collect(java.util.stream.Collectors.toList()));
    }

@GetMapping("/proximas-atividades")
public ResponseEntity<List<Map<String, Object>>> proximasAtividades(
    @RequestParam Integer idTime) {
    LocalDateTime agora = LocalDateTime.now();
    return ResponseEntity.ok(
        atividadeDAO.buscarPorTime(idTime).stream()
            .filter(a->a.getData atividade()!=null
                && a.getData atividade().isAfter(agora)
                && a.getStatus atividade()==1)
            .sorted(java.util.Comparator.comparing(Atividade::getData atividade))
            .limit(5)
            .map(a->Map.of(
                "nome", a.getNome atividade(),
                "tipo", a.getTipo atividade(),
                "data", a.getData atividade().toString(),
                "local", a.getLocal atividade()!=null?a.getLocal atividade():""))
            .collect(java.util.stream.Collectors.toList()));
    }
}

```

14. n8n — Fluxos de Automação

Plataforma low-code que orquestra os fluxos de IA. Rodando em Docker porta 5678. Dois fluxos necessários.

#	Nó	Função
1	Trigger Manual	Inicia o fluxo manualmente
2	Read Binary File	Lê o PDF de uma pasta local fixa
3	Extract from File	Extrai texto do PDF
4	Recursive Character Text Splitter	Divide em chunks ~500 tokens com overlap 50
5	OpenAI Embeddings	text-embedding-3-small → vetor 1536 dims
6	Qdrant — Delete Collection	Apaga coleção 'laivy_pdf' antiga
7	Qdrant — Insert	Insere novos vetores

Fluxo 2 — Responder Perguntas (Webhook ativo)

#	Nó	Função
1	Webhook POST	Recebe { pergunta, idUsuario, idTime, tipo_usuario }
2	IF — valida token	Checa header X-Token. Aborta se inválido
3	OpenAI — classificar	Decide: PDF, dados reais, ou ambos
4a	Qdrant — busca	Embed pergunta → top 5 chunks mais similares
4b	HTTP Request	GET /assistente/resumo-* com dados reais
5	Merge	Combina contexto de PDF e/ou dados reais
6	OpenAI Chat	gpt-4o-mini com prompt + contexto + perfil
7	Respond to Webhook	Retorna { resposta, fonte } para Spring Boot

15. Qdrant — Base Vetorial e RAG

Banco de dados vetorial de alta performance para busca por similaridade semântica. Docker porta 6333. Interface dashboard em <http://localhost:6333/dashboard>

O que é RAG — Retrieval-Augmented Generation

SEM RAG: pergunta → OpenAI → responde com conhecimento geral (pode inventar)

COM RAG:

1. PDF dividido em chunks → embeddings → Qdrant
2. Pergunta → embedding → busca no Qdrant (cosine similarity)
3. Top 5 chunks relevantes retornados
4. Prompt: "Baseado neste contexto: [chunks] responda: [pergunta]"
5. OpenAI responde com base no documento real → sem alucinação

Criar coleção via API REST

```
PUT http://localhost:6333/collections/laivy_pdf
{
  "vectors": { "size": 1536, "distance": "Cosine" }
}
# size=1536 corresponde ao text-embedding-3-small da OpenAI
```

16. Docker e Ambiente Local

laivv/ia/docker-compose.yml

```
version: '3.8'
services:
  n8n:
    image: n8nio/n8n
    container name: laivv-n8n
    ports: ["5678:5678"]
    environment:
      - N8N_BASIC_AUTH_ACTIVE=true
      - N8N_BASIC_AUTH_USER=admin
      - N8N_BASIC_AUTH_PASSWORD=SUA SENHA SEGURA
      - WEBHOOK_URL=http://localhost:5678
    volumes: ["n8n_data:/home/node/.n8n"]
    restart: unless-stopped
  adrant:
    image: adrant/adrant
    container name: laivv-adrant
    ports: ["6333:6333"]
    volumes: ["adrant_data:/adrant/storage"]
    restart: unless-stopped
volumes:
  n8n_data:
  qdrant_data:
```

Serviço	URL	Acesso
LAIVY	http://localhost:8080	Navegador — login.html
n8n	http://localhost:5678	admin / sua_senha
Qdrant	http://localhost:6333/dashboard	Sem autenticação (local)

■ ■ Não subir docker-compose.yml para o GitHub. Adicionar ia/ ao .gitignore.

17. Segurança e Melhorias Planejadas

Item	Status	Detalhe
CORS @CrossOrigin	Implementado	Aceita qualquer origem (*)
Token X-Token Spring→n8n	Implementado	application.properties (fora do frontend)
Dados sensíveis no chat	Implementado	Chat não expõe CPF, senha, dados médicos
docker-compose no .gitignore	Planejado	Evitar expor credenciais no GitHub
BCrypt para senhas	Planejado	BCryptPasswordEncoder via Spring Security
JWT com expiração	Planejado	Substituir localStorage por token seguro
Rate limit no chat	Planejado	Evitar abuso da API OpenAI
HTTPS	Planejado	Certificado SSL para produção

18. Como Executar o Projeto

1. MySQL

```
source banco.sql
# Cria o schema sistema_esportivo e tabelas iniciais
```

2. application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/sistema_esportivo
spring.datasource.username=root
spring.datasource.password=SUA_SENHA
assistente.n8n.webhook-url=http://localhost:5678/webhook/laivv-assistente
assistente.n8n.token=TOKEN_SECRETO
```

3. Docker (n8n + Qdrant)

```
cd laivv/ia
docker compose up -d
docker compose ps
```

4. Spring Boot

```
./mvnw spring-boot:run
# Aguarda: Started ProjetoApplication in X seconds
```

5. Acessar

```
http://localhost:8080/login.html
```

6. n8n (1ª vez)

```
# Acesse http://localhost:5678
# Configure API Key OpenAI em Credentials
# Importe os dois fluxos
# Execute o fluxo de indexação do PDF manualmente
```

19. Perguntas da Banca — Gabarito + Espaço para Respostas

Esta seção contém as perguntas mais prováveis da banca avaliadora, organizadas por tema, com gabarito resumido e espaço para anotações.

Sobre o Projeto em Geral

P1. O que é o LAIVY e qual problema ele resolve?

Gabarito: Sistema de gerenciamento esportivo que centraliza times, atletas, atividades, desempenho e exames em uma única plataforma com controle de acesso por perfil.

Anotações:

P2. Por que escolheram Spring Boot para o backend?

Gabarito: Spring Boot facilita a criação de APIs REST com configuração mínima, tem integração nativa com JPA/Hibernate e Tomcat embutido, permitindo deploy simples como JAR ou WAR.

Anotações:

P3. Qual é a diferença entre os três perfis de usuário?

Gabarito: Admin (tipo=1): acesso total. Treinador (tipo=2): gerencia seu time. Atleta (tipo=3): visualiza dados próprios e do time.

Anotações:

P4. O sistema tem alguma falha de segurança? Como corrigiram ou planejam corrigir?

Gabarito: Sim — senhas em texto plano e sem JWT. Melhoria planejada: BCrypt e Spring Security com JWT.

Anotações:

Sobre a Arquitetura

P1. Por que não usaram uma camada de Service entre Controller e DAO?

Gabarito: Para o escopo do TCC, a lógica de negócio é simples e os controllers gerenciam diretamente as operações. Em sistemas maiores, uma camada Service centralizaria regras e facilitaria testes unitários.

Anotações:

P2. O que é JPA e qual a vantagem sobre SQL puro?

Gabarito: JPA (Java Persistence API) abstrai o banco: você trabalha com objetos Java. O Hibernate gera o SQL automaticamente. Vantagem: menos código, portabilidade entre bancos e ddl-auto=update que cria/atualiza tabelas automaticamente.

Anotações:

P3. O que é o ddl-auto=update e quais riscos ele tem?

Gabarito: Faz o Hibernate atualizar o schema automaticamente na inicialização. Risco em produção: pode alterar tabelas acidentalmente. Em produção, usar validate ou none.

Anotações:

P4. Como o frontend se comunica com o backend?

Gabarito: Via fetch() JavaScript, fazendo chamadas HTTP para a API REST. O Spring Boot serve os HTMLs estáticos em /static/ e a API no mesmo servidor.

Anotações:

Sobre o Banco de Dados

P1. Quais são as principais relações entre as tabelas?

Gabarito: Usuario→Time (ManyToOne), Time→Usuario/treinador (ManyToOne), Atividade→Time (ManyToOne), Escalacao→Atividade+Usuario, Calendario→Atividade, Visitante→Calendario.

Anotações:

P2. Por que usaram MEDIUMTEXT para foto e logo?

Gabarito: As imagens são armazenadas como Base64, que pode ter vários KB. VARCHAR tem limite de 65.535 bytes; MEDIUMTEXT suporta até 16MB.

Anotações:

P3. O que é uma @Query JPQL e por que usaram em vez de métodos automáticos do Spring Data?

Gabarito: JPQL usa nomes de classes Java, não tabelas. Usamos @Query quando o nome do campo não segue a convenção Spring Data (ex: ativo_time com underline).

Anotações:

Sobre o Chat Inteligente / IA

P1. O que é RAG?

Gabarito: Retrieval-Augmented Generation. Em vez de a IA responder do nada, ela busca trechos do PDF por similaridade semântica no Qdrant e responde baseada nesses trechos. Evita alucinações.

Anotações:

P2. Para que serve o n8n no projeto?

Gabarito: Orquestra o fluxo de IA: recebe a pergunta, decide a fonte (PDF ou dados reais), faz as buscas, monta o prompt e chama a OpenAI. Funciona como middleware de IA.

Anotações:

P3. Por que o frontend não chama o webhook do n8n diretamente?

Gabarito: Segurança: a URL e o token secreto ficam no application.properties do Spring Boot, não expostos no código frontend do navegador.

Anotações:

P4. O que é um embedding?

Gabarito: Uma representação numérica (vetor) do significado de um texto. Textos com significado similar têm vetores próximos. O Qdrant busca por proximidade vetorial.

Anotações:

P5. Qual modelo de IA vocês usaram e por quê?

Gabarito: text-embedding-3-small para embeddings (barato, 1536 dims). gpt-4o-mini para respostas (custo-benefício alto, rápido).

Anotações:

Sobre o Frontend

P1. Por que usaram HTML puro em vez de React ou Angular?

Gabarito: Simplicidade para o escopo do TCC. HTML servido pelo Spring Boot não precisa de build tool, CDN ou configuração extra. A equipe tem mais domínio de HTML/JS vanilla.

Anotações:

P2. Como a sessão do usuário é mantida sem backend?

Gabarito: Via localStorage. O objeto do usuário logado é salvo como JSON após o login e lido em cada tela. A sessão é limpa no logout.

Anotações:

P3. O que é o FullCalendar e como vocês o integraram?

Gabarito: Biblioteca JS de calendário interativo. Carregamos os eventos via API (GET /atividades?idTime=N) e formatamos como objetos { title, start, color } para o FullCalendar renderizar.

Anotações:

Anotações Livres — Página 1/3

Use este espaço para anotações durante a apresentação.

Anotações Livres — Página 2/3

Use este espaço para anotações durante a apresentação.

Use este espaço para anotações durante a apresentação.